Project Report

Comparison between PostgreSQL and Neo4j for AI Research Paper Retrieval and Knowledge Graph Analysis

Data Management 2025/2026

Students	Hamza Abdul Kader, matricola 2230150
	Leonardo Marasca, matricola 2217140

Project type	NoSQL / DBMS comparison
Dataset	OpenAlex AI research paper subset
Systems compared	PostgreSQL 16 and Neo4j 5

This report is the polished PDF version of the living project documentation.

Contents

1 Executive Summary

This project compares a relational database management system and a graph database using the same dataset of AI-related research papers collected from OpenAlex. The relational implementation uses PostgreSQL, while the graph implementation uses Neo4j.

The main objective is not only to check which system is faster, but also to understand how each system represents and queries relationship-heavy scholarly data. Scientific papers naturally form a graph: authors write papers, papers cite other papers, papers have topics, and authors become connected through shared publications, topics, and citations.

The work completed so far includes:

- collecting and cleaning an OpenAlex subset focused on AI, Machine Learning, Large Language Models, and Retrieval-Augmented Generation;
- modeling the same data in PostgreSQL and Neo4j;
- loading both databases with the same CSV files;
- validating that both databases contain matching entity and relationship counts;
- writing equivalent SQL and Cypher queries;
- benchmarking 11 query pairs;
- generating charts and tables for presentation.

The current benchmark shows that PostgreSQL is faster on most queries for this small dataset, especially ranking and aggregation queries. Neo4j is faster on the author citation network query, where the traversal follows a natural author-to-paper-to-paper-to-author graph pattern. To strengthen the evaluation, an additional graph-focused workload was also added. This supplemental workload shows that Neo4j gains further advantages when the query combines multi-hop citation traversal with network-style relationship constraints. The broader conclusion is that PostgreSQL is very efficient for tabular analysis, while Neo4j is often clearer and more direct for graph-shaped questions.

2 Project Goal and Research Questions

The goal of the project is to compare PostgreSQL and Neo4j on the same AI research paper dataset. The comparison focuses on data modeling, query complexity, expressiveness, execution time, and suitability for relationship-heavy analysis.

The main research questions are:

1. How can the same scholarly dataset be modeled in a relational database and in a graph database?
2. Which system makes common analytical queries easier to express?
3. Which system performs better on the selected dataset and query workload?
4. Are graph-shaped queries more naturally represented in Neo4j than in PostgreSQL?
5. What are the practical trade-offs between relational joins and graph traversals?

3 Dataset

3.1 Source

The dataset source is OpenAlex, an open catalog of scholarly works, authors, institutions, topics, citations, and related academic metadata.

- Main website: `https://openalex.org/`
- Dataset documentation: OpenAlex About the data

OpenAlex is appropriate for this project because it already contains the types of entities and relationships that are useful for comparing relational and graph databases:

- papers and their metadata;
- authorship relationships;
- topics and concepts;
- citation links between papers;
- publication years, titles, abstracts, and citation counts.

3.2 Scope of the Collected Subset

The full OpenAlex dataset is too large for a student project, so we collect a focused subset through the OpenAlex API. The selected subset is related to:

- retrieval augmented generation;
- large language models;
- artificial intelligence;
- machine learning.

The current sample was collected with:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

Table 1: Current OpenAlex dataset sample

Entity	**Count**
Papers	299
Authors	1966
Topics	1104
Paper-author relationships	2077
Paper-topic relationships	4288
Citation relationships	392

Generated data files are stored locally in `data/raw/` and `data/processed/`. They are ignored by git because they can be regenerated from the OpenAlex API.

This dataset is suitable for a DBMS comparison because it contains both tabular attributes and graph-shaped relationships. For example, paper metadata such as title, year, DOI, and citation count fits naturally in relational columns, while authorship, topic assignment, and citations form connected paths. This makes it possible to compare not only execution time, but also modeling clarity.

4 Data Collection and Cleaning Pipeline

The collection script is `src/collect_openalex.py`. It uses only the Python standard library so the project can run without dependency installation.

The pipeline performs the following operations:

1. Calls the OpenAlex Works API using AI-related search terms.
2. Uses cursor pagination to retrieve multiple pages of results.
3. Saves raw API objects as JSONL.
4. Reconstructs abstracts from OpenAlex abstract inverted indexes when available.
5. Extracts paper metadata such as title, year, DOI, OpenAlex ID, citation count, and matched search terms.
6. Extracts authors and paper-author relationships.
7. Extracts topics and paper-topic relationships.
8. Extracts citations where both the citing and cited papers are inside the selected subset.
9. Removes duplicate paper-author pairs before database loading.
10. Writes normalized CSV files for both PostgreSQL and Neo4j.

Table 2: Generated CSV files

File	**Content**
`papers.csv`	One row per selected OpenAlex work.
`authors.csv`	One row per author.
`topics.csv`	One row per topic or concept.
`paper_authors.csv`	Many-to-many relationship between papers and authors.
`paper_topics.csv`	Many-to-many relationship between papers and topics.
`citations.csv`	Citation links where both papers are inside the selected subset.

4.1 Data Quality Issue

During PostgreSQL loading, the project found that OpenAlex can contain duplicate author entries for the same paper. This produced a duplicate primary key error in the `paper_authors` table.

The fix was to de-duplicate paper-author relationships by `(paper_id, author_id)` during normalization. This is a useful project detail because it shows a realistic data cleaning problem and how it was solved.

5 Repository Structure

```
src/
  collect_openalex.py
  benchmark_queries.py
  generate_benchmark_charts.py

database/postgres/
  schema.sql
  load.sql
  queries.sql

database/neo4j/
  constraints.cypher
  import.cypher
  queries.cypher

benchmarks/
  queries.json
  results/benchmark_results.csv

docs/
  project_proposal.pdf
  project_roadmap.pdf
  project_report.md
  project_report.tex
  project_report.pdf
  benchmark_results.md
  figures/
```

6 PostgreSQL Relational Model

PostgreSQL uses a normalized relational schema. Entities are represented as tables, and many-to-many relationships are represented as bridge tables.

Table 3: PostgreSQL tables

Table	**Purpose**
`papers`	Stores paper metadata such as title, year, DOI, abstract, topic, citation count, and OpenAlex URL.
`authors`	Stores author IDs and display names.
`topics`	Stores topic IDs and names.
`paper_authors`	Bridge table connecting papers and authors.
`paper_topics`	Bridge table connecting papers and topics.
`citations`	Stores directed paper-to-paper citation links.

This model is strong for:

- aggregations;

- filtering;
- sorting;
- structured joins;
- enforcing primary and foreign keys.

However, graph-shaped questions can become verbose because they require multiple joins or recursive queries.

7 Neo4j Graph Model

Neo4j uses nodes and relationships. This representation is closer to the natural structure of scholarly data.

Table 4: Neo4j graph model

Graph element	**Meaning**
`(:Paper)`	Research paper node.
`(:Author)`	Author node.
`(:Topic)`	Topic node.
Author → Paper	Authorship relationship, implemented as `AUTHORED`.
Paper → Topic	Topic assignment relationship, implemented as `HAS_TOPIC`.
Paper → Paper	Directed citation relationship, implemented as `CITES`.

Example graph patterns:

```
(Author)-[:AUTHORED]->(Paper)
(Paper)-[:HAS_TOPIC]->(Topic)
(Paper)-[:CITES]->(Paper)
```

This model is strong for:

- citation paths;
- author collaboration;
- shared topics;
- author citation networks;
- relationship traversal.

8 Database Setup and Loading

The project uses Docker Compose to run both systems locally.

Adminer is included only as a visual interface for PostgreSQL. It does not change the benchmark; it helps inspect tables and execute SQL queries during the demo.

Start the databases:

```
docker compose up -d
```

Table 5: Docker services

Service	Image	Access
PostgreSQL	`postgres:16`	`localhost:5432`
Neo4j	`neo4j:5`	`http://localhost:7474`
Adminer	`adminer:latest`	`http://localhost:8080`

Load PostgreSQL:

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

Load Neo4j:

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
  -f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
  -f /cypher/import.cypher
```

9 Validation

After loading both databases, the counts were compared to ensure that the same dataset was present in both systems.

Table 6: Validation of loaded data

Data element	PostgreSQL	Neo4j
Papers	299	299
Authors	1966	1966
Topics	1104	1104
Paper-author / AUTHORED	2077	2077
Paper-topic / HAS_TOPIC	4288	4288
Citations / CITES	392	392

The counts match exactly, confirming that the benchmark compares the same logical data in both systems.

10 Visual Evidence and Demo Screenshots

This section documents the visual evidence used in the final presentation. The screenshots are not used as benchmark inputs; they show that the databases were actually loaded and inspected through their user interfaces.

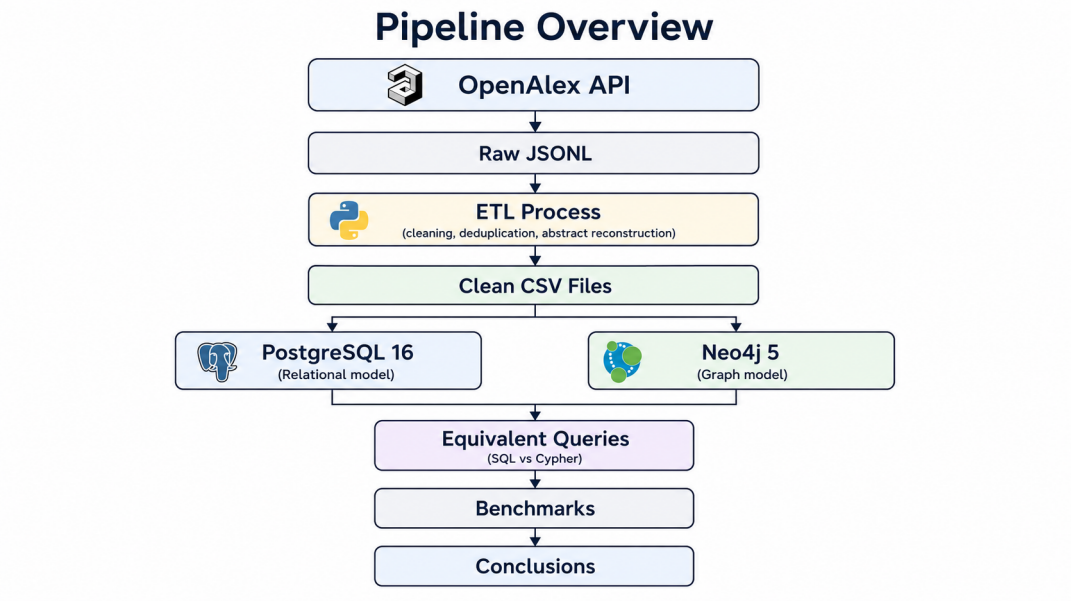

Figure 1: End-to-end project pipeline from OpenAlex API collection to benchmark conclusions

Figure 1 summarizes the complete workflow. The same data is collected once from OpenAlex, normalized once into CSV files, and then loaded into both database systems. This is important because the comparison would not be fair if PostgreSQL and Neo4j were loaded from different datasets.

```
PS C:\Users\Lenovo\Desktop\Sperenza\Spring_2026\Data_Managment\project> docker compose ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED          STATUS          PORTS
openalex-adminer    adminer:latest     "entrypoint.sh docke…"   adminer    2 minutes ago    Up 2 minutes    0.0.0.0:8080->80
80/tcp, [::]:8080->8080/tcp
openalex-neo4j      neo4j:5            "tini -g -- /startup…"   neo4j      2 minutes ago    Up 2 minutes    0.0.0.0:7474->74
74/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp
openalex-postgres   postgres:16        "docker-entrypoint.s…"   postgres   2 minutes ago    Up 2 minutes    0.0.0.0:5432->54
32/tcp, [::]:5432->5432/tcp
PS C:\Users\Lenovo\Desktop\Sperenza\Spring_2026\Data_Managment\project>
```

Figure 2: Docker Compose services running locally

Figure 2 shows the local Docker environment. The running services are PostgreSQL, Neo4j, and Adminer. This confirms that the demo environment can be started with a single Docker Compose command.

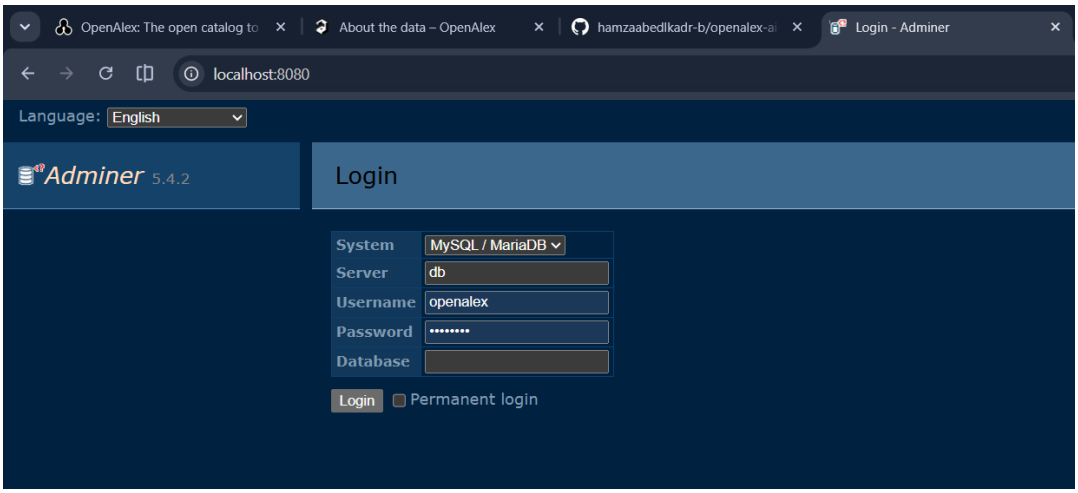

Figure 3: PostgreSQL tables visible in Adminer

Figure 3 shows the PostgreSQL database through Adminer. The table list corresponds to the relational model: entity tables for papers, authors, and topics, plus bridge tables and citations.

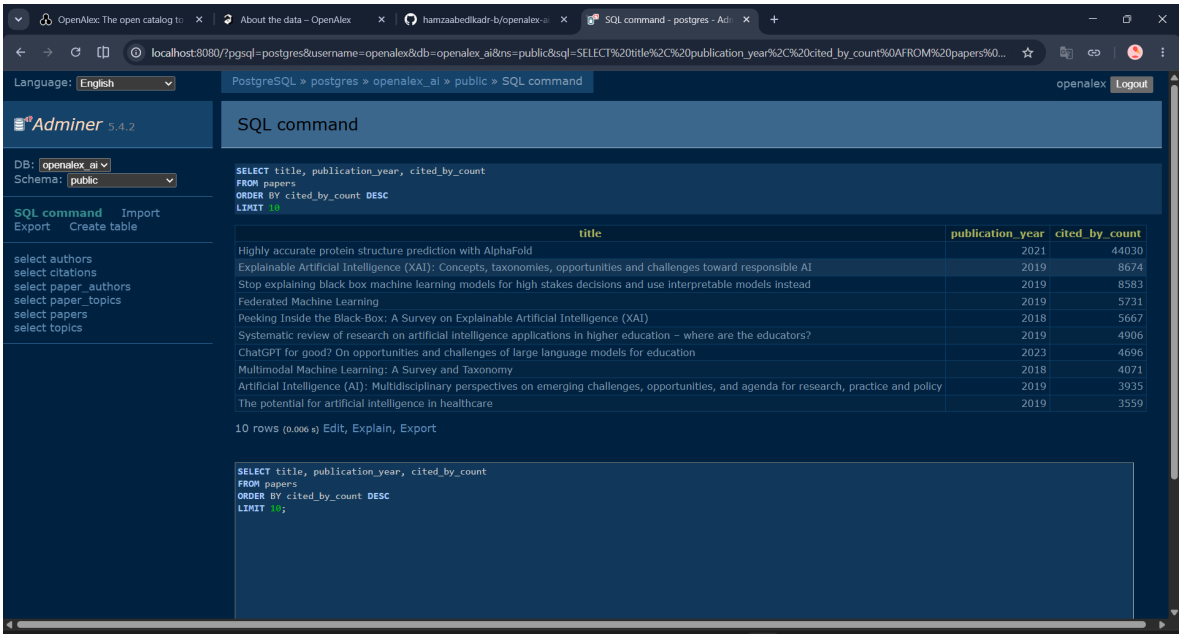

Figure 4: Example SQL query result in PostgreSQL

Figure 4 shows a SQL query executed against the loaded PostgreSQL database. This evidence connects the schema to an actual analytical query result.

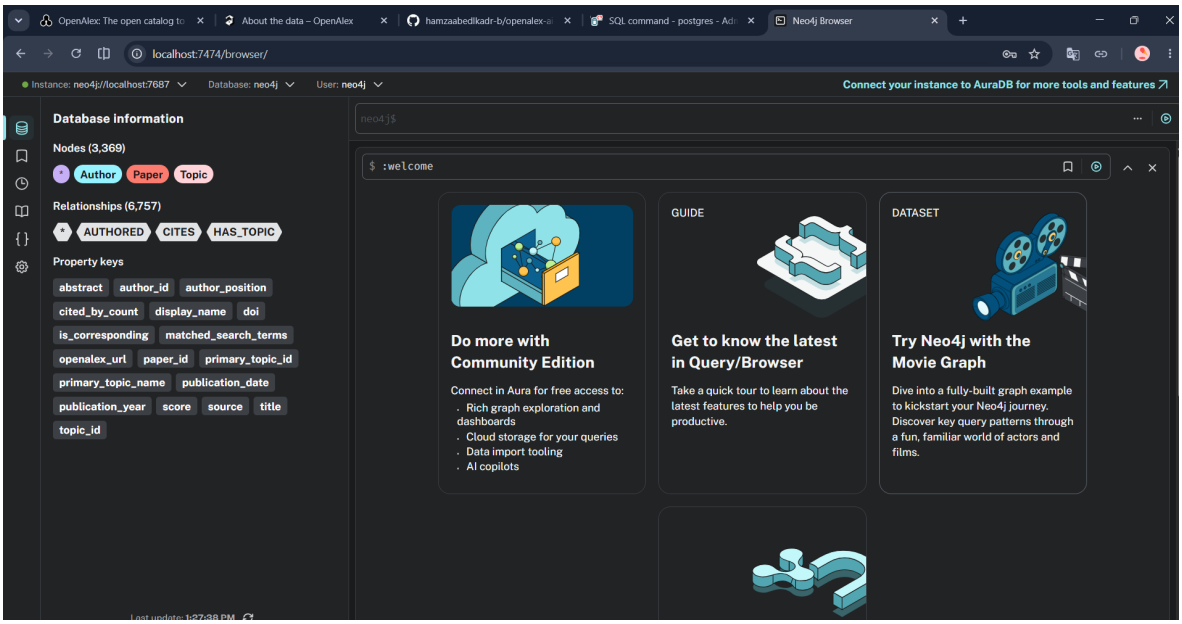

Figure 5: Neo4j Browser overview with node labels and relationship types

Figure 5 shows that Neo4j contains the expected node labels `Author`, `Paper`, and `Topic`, and the expected relationship types `AUTHORED`, `HAS_TOPIC`, and `CITES`.

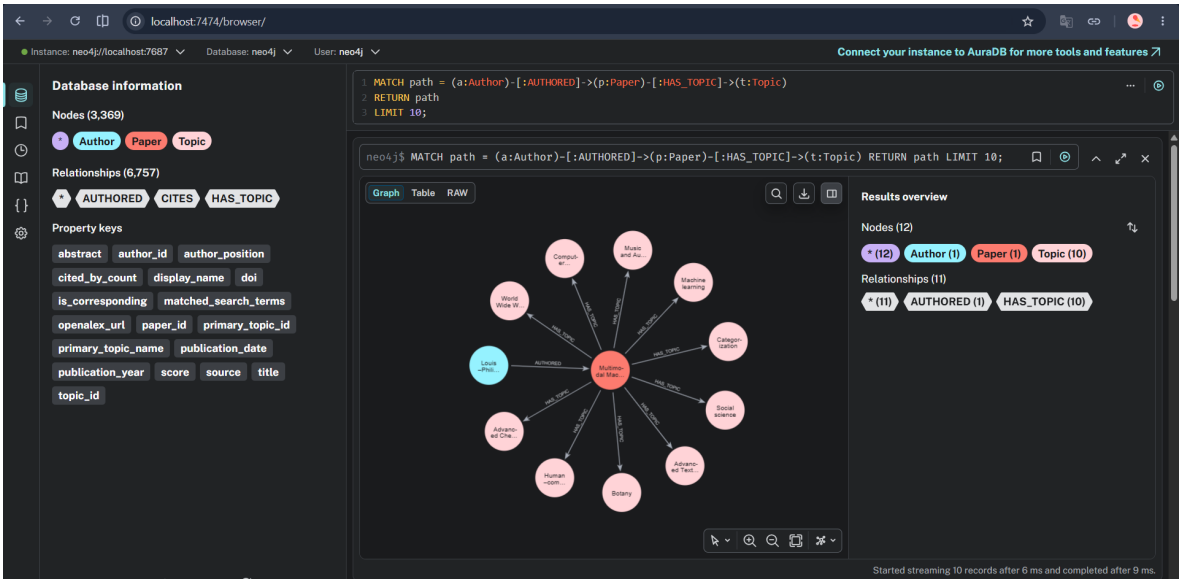

Figure 6: Neo4j graph query for Author–Paper–Topic paths

Figure 6 shows an example graph pattern where an author wrote a paper and the paper has topics. The query returns a path, so Neo4j Browser displays both nodes and relationships.

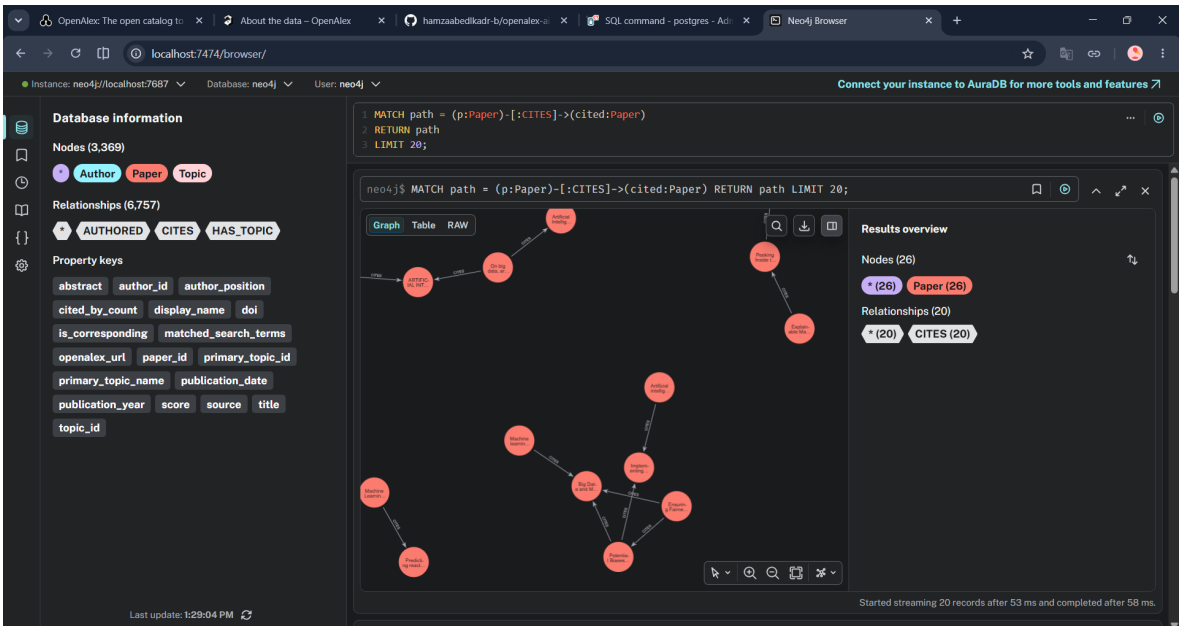

Figure 7: Neo4j graph query for paper citation relationships

Figure 7 shows the `CITES` relationship between paper nodes. This is one of the clearest examples of why a graph model is useful for scholarly data: citation networks can be inspected as connected objects rather than only as rows of IDs.

11 Query Workload

The benchmark contains 11 paired SQL/Cypher queries. They are stored in `benchmarks/queries.json`.

Table 7: Benchmark query workload

ID	Query	Purpose
Q1	Most cited papers	Rank papers by citation count.
Q2	Authors with the most papers	Count selected papers per author.
Q3	Most frequent topics	Count selected papers per topic.
Q4	Author collaboration pairs	Find authors who worked together most often.
Q5	Citation links inside the subset	Return directed citation relationships.
Q6	RAG-related papers	Search title and abstract for RAG-related terms.
Q7	Two-hop citation paths	Find papers connected by a two-step citation path.
Q8	Authors connected through shared topics	Find author pairs with many shared topics.
Q9	Papers sharing cited references	Find papers that cite the same references.
Q10	Citation paths up to three hops	Find paper pairs connected by citation paths of length two or three.
Q11	Author citation network	Find authors connected when one author's papers cite another author's papers.

12 Example of Equivalent SQL and Cypher

The benchmark pairs each analytical question with one SQL query and one Cypher query. The goal is not to make the queries textually identical, but to make them answer the same logical question over the same loaded data.

One representative example is the author citation network query. The question is: *which authors are connected because one author's paper cites another author's paper?*

In Neo4j, the graph path is written directly:

```
MATCH (source:Author)-[:AUTHORED]->(:Paper)-[:CITES]->(:Paper)
      <-[:AUTHORED]-(target:Author)
WHERE source.author_id <> target.author_id
RETURN source.display_name AS citing_author,
       target.display_name AS cited_author,
       count(*) AS citation_edges
ORDER BY citation_edges DESC
LIMIT 10;
```

The equivalent SQL query follows the same logical path using joins:

```
SELECT
    source_author.display_name AS citing_author,
    target_author.display_name AS cited_author,
    COUNT(*) AS citation_edges
FROM paper_authors source_pa
JOIN citations c ON c.citing_paper_id = source_pa.paper_id
JOIN paper_authors target_pa ON target_pa.paper_id = c.cited_paper_id
JOIN authors source_author ON source_author.author_id = source_pa.author_id
JOIN authors target_author ON target_author.author_id = target_pa.author_id
WHERE source_pa.author_id <> target_pa.author_id
GROUP BY source_author.author_id, source_author.display_name,
         target_author.author_id, target_author.display_name
ORDER BY citation_edges DESC
LIMIT 10;
```

This example shows the main modeling difference. In Neo4j, the query is shaped like the relationship path. In PostgreSQL, the same path is reconstructed through relational joins.

13 Benchmark Methodology

The benchmark is run with:

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

The methodology is:

- one warmup run per query/system pair;
- five measured runs per query/system pair;
- PostgreSQL timing uses `EXPLAIN (ANALYZE, FORMAT JSON)`;
- Neo4j timing uses `PROFILE` through `cypher-shell --format verbose`;
- PostgreSQL planning time and Neo4j database-access counts are also recorded where available;
- raw results are saved in `benchmarks/results/benchmark_results.csv`;
- Markdown results are saved in `docs/benchmark_results.md`.

This benchmark should be interpreted as a local project benchmark, not as a universal ranking of PostgreSQL and Neo4j. Performance depends on data size, indexing, cache state, hardware, and query shape.

14 Benchmark Results

Table 8: Average benchmark execution time

Query	**PostgreSQL avg ms**	**Neo4j avg ms**	**Faster system**
Most cited papers	0.058	4.200	PostgreSQL
Authors with the most papers	2.422	8.600	PostgreSQL
Most frequent topics	4.762	15.200	PostgreSQL
Author collaboration pairs	44.358	65.000	PostgreSQL
Citation links inside the subset	0.183	4.400	PostgreSQL
RAG-related papers	3.515	8.600	PostgreSQL
Two-hop citation paths	1.577	8.400	PostgreSQL
Authors connected through shared topics	309.447	586.600	PostgreSQL
Papers sharing cited references	2.171	7.800	PostgreSQL
Citation paths up to three hops	5.104	12.400	PostgreSQL
Author citation network	48.214	37.400	Neo4j

14.1 Result Summary

The benchmark has one clear numerical result: PostgreSQL is faster on 10 of the 11 benchmarked query pairs. The one exception is the author citation network query, where Neo4j is faster.

Table 9: Summary of benchmark wins

System	**Queries won**	**Main reason**
PostgreSQL	10	The current dataset is small and many queries are structured aggregations, joins, filters, and sorts. These operations match PostgreSQL's relational execution engine well.
Neo4j	1	The author citation network query follows a natural graph path from author to paper to cited paper to author. Cypher expresses this traversal directly.

This does not mean that PostgreSQL is always better than Neo4j. It means that, for this dataset size and this workload, many questions are closer to traditional analytical SQL than to deep graph traversal. Neo4j becomes more attractive when the question asks about paths, neighborhoods, influence, communities, or networks.

14.2 Operator-Level Interpretation

The benchmark was improved by looking beyond average runtime. PostgreSQL reports planning and execution information through `EXPLAIN ANALYZE`, while Neo4j reports database-access counts through `PROFILE`. These extra metrics help connect the benchmark to the course topic of query processing and operator evaluation.

Table 10: Operator-level interpretation of representative queries

Query type	**PostgreSQL behavior**	**Neo4j behavior**
Ranking and filtering	Uses scans, indexes, sort, and limit operators. On the small dataset, these operations are very cheap.	Often starts with node scans or label scans, then sorts properties. The fixed overhead is visible for small result sets.
Relational aggregations	Uses joins followed by grouping and ordering. PostgreSQL is highly optimized for this pattern.	Uses graph expansion followed by aggregation. This is readable in Cypher, but may touch more graph records.
Shared-topic network query	Requires many joins across author, paper, topic, and bridge tables. Planning and execution cost increase strongly.	Traverses explicit `AUTHORED`, `CITES`, and `HAS_TOPIC` relationships. Even with many database hits, the path expression maps naturally to the graph.
Author citation network	Reconstructs the path through several joins over bridge tables and citations.	Traverses the stored path `Author -> Paper -> Paper -> Author`, which is the natural graph shape.

For example, the original author citation network query averages 48.214 ms in PostgreSQL and 37.400 ms in Neo4j. The PostgreSQL version rebuilds the author-to-paper-to-cited-paper-to-author path through joins. The Neo4j version traverses the same path directly through relationships.

Another useful example is the original shared-topics query. It is slow in both systems because it compares many author/topic combinations. PostgreSQL averages 309.447 ms, while Neo4j averages 586.600 ms and reports 1,845,747 database accesses. This shows that graph syntax alone does not guarantee faster execution: if a query expands a very large relationship space, the graph engine still has a lot of work to do.

14.3 Supplemental Graph-Focused Workload

To make the Neo4j evaluation stronger, a supplemental graph-focused workload was added in:

`benchmarks/graph_focused_queries.json`

This second workload does not replace the original benchmark. It is used to test whether Neo4j becomes more competitive when the questions are more explicitly graph-shaped. The measured results are saved in:

`benchmarks/results/graph_focused_benchmark_results.csv`
`docs/graph_focused_benchmark_results.md`

Table 11: Supplemental graph-focused benchmark results

Query	PostgreSQL avg ms	Neo4j avg ms	Faster system
Anchored author citation neighborhood	0.456	1.400	PostgreSQL
Paper citation neighborhood within two hops	0.441	2.800	PostgreSQL
Paper context subgraph	0.605	4.600	PostgreSQL
Topic-to-author neighborhood	0.694	2.600	PostgreSQL
Topic-filtered author citation network	0.563	3.800	PostgreSQL
Two-hop author citation network	43.028	39.200	Neo4j
Author citation with shared paper topic	783.048	338.000	Neo4j

The supplemental workload gives a more nuanced result. PostgreSQL still wins the small anchored neighborhood queries because the dataset is small and indexed relational lookups are extremely cheap. However, Neo4j wins two more network-style queries:

- **Two-hop author citation network**: Neo4j averages 39.200 ms versus PostgreSQL's 43.028 ms.
- **Author citation with shared paper topic**: Neo4j averages 338.000 ms versus PostgreSQL's 783.048 ms.

This improves the interpretation of Neo4j's role in the project. Neo4j is not faster for every graph-looking query. It becomes more competitive when the query requires multi-hop traversal and combines several relationship types, such as authorship, citation, and topic overlap.

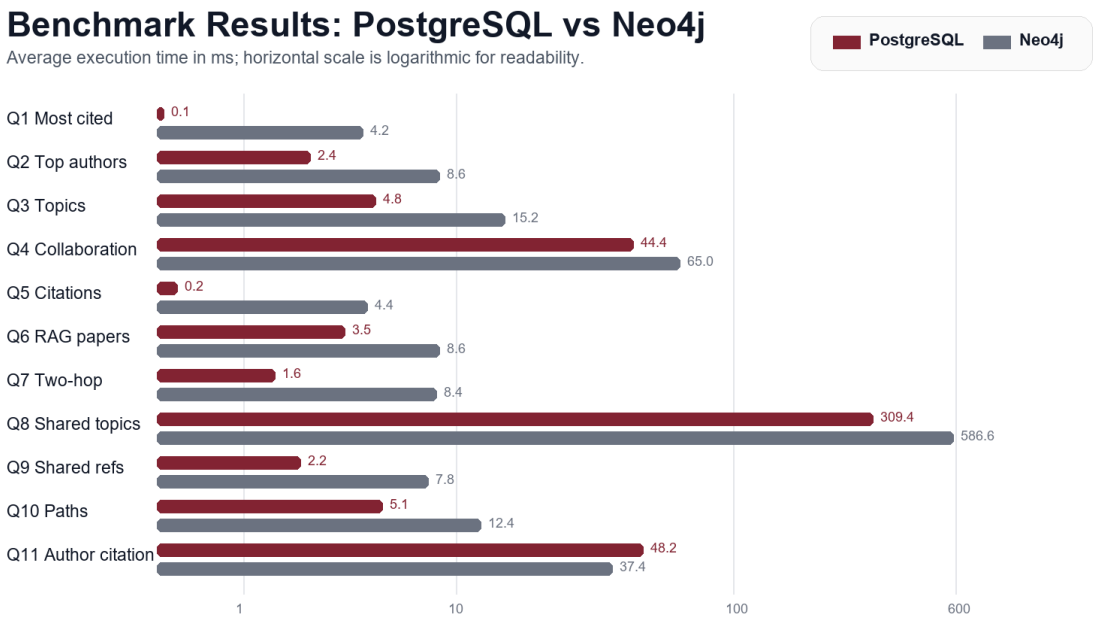

Figure 8: Presentation benchmark chart generated from the measured result CSV

Figure 8 is the simplified chart used in the final presentation. The chart is generated from the same benchmark CSV file as Table 8.

15 Benchmark Graphs

Figure 9: Average query time comparison

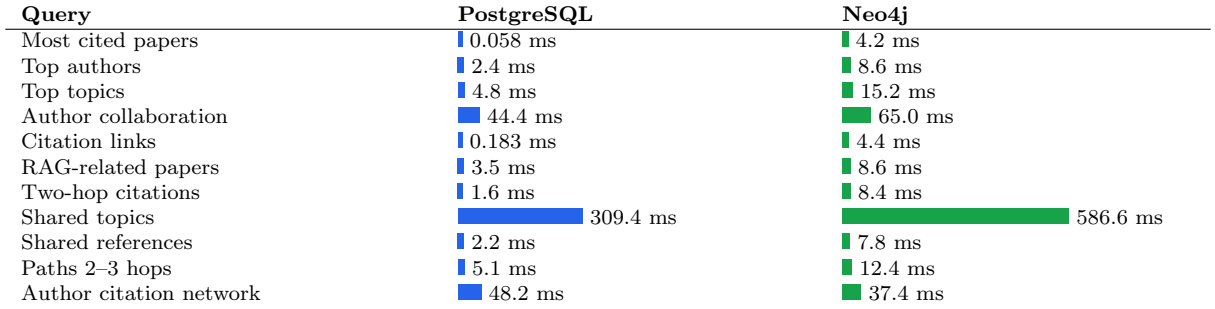

Figure 10: Relative query time: Neo4j average divided by PostgreSQL average

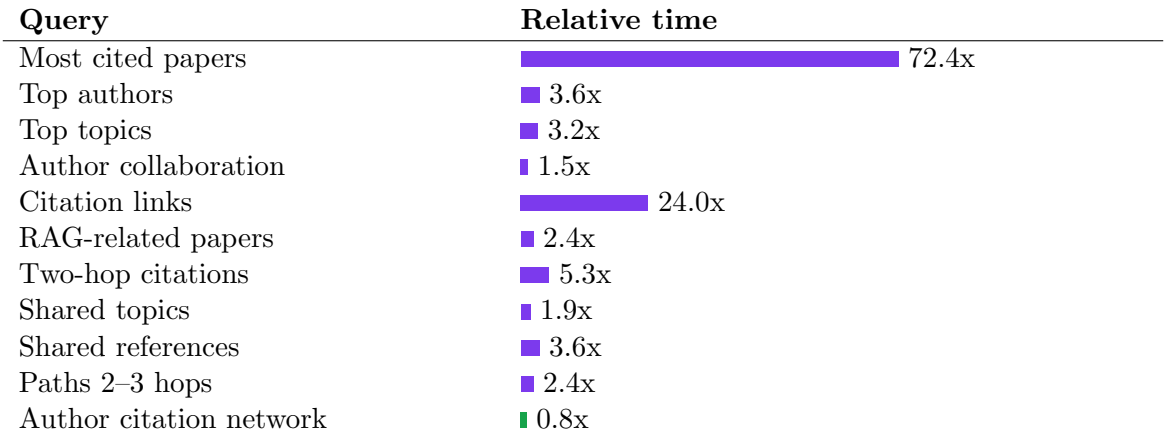

Figure 9 shows absolute average execution time. Figure 10 shows how many times slower or faster Neo4j was relative to PostgreSQL. Values above 1x mean Neo4j was slower than PostgreSQL; values below 1x mean Neo4j was faster.

16 Interpretation

The benchmark has three important interpretations.

16.1 PostgreSQL is strong for tabular aggregation

Queries such as most cited papers, most frequent topics, and citation links are very efficient in PostgreSQL. These queries match the strengths of a relational DBMS: indexes, joins, grouping, and ordering.

16.2 Neo4j is clearer for graph-shaped questions

Neo4j's Cypher syntax is often easier to read when the question is naturally expressed as a path. For example, an author citation network can be expressed as:

```
(Author)-[:AUTHORED]->(Paper)-[:CITES]->(Paper)<-[:AUTHORED]-(Author)
```

In PostgreSQL, the same query requires several joins across `paper_authors`, `citations`, and `authors`.

16.3 The result depends on dataset size and query shape

The current dataset is intentionally small. Therefore, the results should not be interpreted as a general statement that PostgreSQL is always faster. With larger graphs, deeper traversals, more complex path search, or additional graph indexes/projections, the comparison could change.

17 Connection to Course Material

The project is connected to several topics discussed in the Data Management course. The connection is not only theoretical: the course concepts directly influenced the way the data was modeled, loaded, queried, and evaluated.

Table 12: Course concepts reflected in the project

Course topic	**Concept**	**How it appears in the project**
Graph databases	Relationships can be represented as explicit edges instead of being reconstructed through joins.	Neo4j stores `AUTHORED`, `HAS_TOPIC`, and `CITES` as relationships.
Neo4j property graph model	Nodes have labels and properties; relationships have types and connect nodes.	We use `Paper`, `Author`, and `Topic` nodes with properties such as IDs, titles, years, and citation counts.
Cypher query language	Graph queries are expressed as path patterns using `MATCH`.	Queries such as author citation networks are written as paths: author → paper → cited paper → author.
Relational model and joins	Relationships in a relational schema are represented through foreign keys and join tables.	PostgreSQL uses `paper_authors`, `paper_topics`, and `citations` to reconstruct relationships through SQL joins.
Query processing and operator evaluation	SQL queries are evaluated through operators such as selection, projection, join, grouping, sorting, and aggregation.	Benchmark queries include filters, joins, `GROUP BY`, `ORDER BY`, and recursive/path-like logic. PostgreSQL timing is measured with `EXPLAIN ANALYZE`.
File organization and indexes	Access paths and indexes influence query cost.	PostgreSQL schema defines primary keys and indexes; Neo4j uses uniqueness constraints and indexes for node lookup before traversal.

17.1 Relational Joins vs Graph Traversal

One of the central course ideas is that relationship-heavy data can be handled in different ways. In a relational DBMS, a relationship is normally reconstructed at query time using joins. In a graph DBMS,

a relationship is stored as a first-class edge and can be traversed directly.

Our project demonstrates this difference with citation and authorship data. For example, the author citation network query has the following logical structure:

```
Author -> Paper -> Cited Paper -> Author
```

In PostgreSQL, this path is rebuilt by joining `paper_authors`, `citations`, and `authors`. In Neo4j, the same path is expressed directly in Cypher. This is why the project is a practical example of the lecture distinction between relationships-as-joins and relationships-as-edges.

17.2 Why RDF and Data Warehousing Were Not Used

The course also covers RDF databases and data warehousing. Those topics are related to data management, but they are not the project model we selected.

RDF represents data as triples and is useful for semantic web and ontology-oriented data. Our implementation uses Neo4j's property graph model instead, because the approved project topic was a DBMS comparison between a relational database and a graph database.

Data warehousing focuses on analytical schemas such as star schemas and OLAP analysis. Our project is not a data warehouse project: we did not design a fact table, dimensions, or OLAP sessions. Instead, we compare PostgreSQL and Neo4j on the same operational/analytical OpenAlex subset. This distinction is important because the evaluation criteria are different.

18 Main Findings

- The same OpenAlex dataset was successfully represented in both PostgreSQL and Neo4j.
- PostgreSQL and Neo4j were loaded with matching entity and relationship counts.
- PostgreSQL was faster on 10 out of 11 benchmarked queries.
- Neo4j was faster on the author citation network query.
- A supplemental graph-focused workload added two more Neo4j wins on multi-hop/network-style queries.
- Neo4j query syntax is often more natural for relationship traversal.
- PostgreSQL is highly efficient for relational aggregation and sorting.
- The most useful comparison is not only speed, but also modeling clarity and query expressiveness.

19 Limitations

- The dataset is a selected subset, not the full OpenAlex dataset.
- Benchmark results were produced on a local machine and depend on local hardware.
- The main benchmark contains 11 queries, plus a smaller supplemental graph-focused workload.
- The benchmark focuses on server-reported query time, not full application-level latency.

- The Neo4j model uses the basic graph database; advanced graph data science projections were not used.

20 Future Work

- Increase the dataset size and rerun the benchmark.
- Add deeper citation path queries.
- Add more RAG-specific retrieval queries.
- Add visual graph exploration examples in Neo4j Browser.
- Add an application layer that uses the databases for a small RAG-style search demo.

21 Final Deliverables

The project repository contains the following final deliverables:

- `docs/project_proposal.pdf`: approved project proposal;
- `docs/project_roadmap.pdf`: project roadmap;
- `docs/project_report.pdf`: detailed final report;
- `docs/OpenAlex_DBMS_Comparison_Sapienza.pptx`: presentation deck;
- `README.md`: run instructions and demo instructions;
- `src/`: data collection, benchmark, and chart generation scripts;
- `database/postgres/`: PostgreSQL schema, load script, and SQL queries;
- `database/neo4j/`: Neo4j constraints, import script, and Cypher queries;
- `benchmarks/results/benchmark_results.csv`: raw benchmark measurements.

22 Conclusion

This project demonstrates a practical comparison between PostgreSQL and Neo4j using AI research paper data from OpenAlex. The relational model is efficient and structured, while the graph model is expressive and natural for relationship-heavy analysis.

The current results show PostgreSQL as faster for most queries on the small benchmark dataset, but Neo4j provides clearer graph traversal semantics and outperforms PostgreSQL on the author citation network query. The supplemental graph-focused workload adds two more Neo4j wins, especially when the query combines citation traversal with topic overlap. This supports a balanced conclusion: PostgreSQL is a strong choice for structured analytical workloads, while Neo4j becomes especially valuable when the main questions are about relationships, paths, and networks.